TASK 1 – Initial API Setup

Prompt:

Generate a basic Flask API with a /status endpoint returning JSON response.

Code:

```python
from flask import Flask, jsonify

app = Flask(__name__)

@app.route('/status', methods=['GET'])
def status():
    return jsonify({"status": "API is running"})

if __name__ == '__main__':
    app.run(debug=True)
```

```
 * Serving Flask app '__main__'
 * Debug mode: on
INFO:werkzeug:WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on http://127.0.0.1:5000
INFO:werkzeug:Press CTRL+C to quit
INFO:werkzeug: * Restarting with watchdog (inotify)
```

Output:

```
 * Serving Flask app '__main__'
 * Debug mode: on
INFO:werkzeug:WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on http://127.0.0.1:5000
INFO:werkzeug:Press CTRL+C to quit
INFO:werkzeug: * Restarting with watchdog (inotify)
```

Justification:

- Confirms backend availability
- Baseline endpoint for health check

TASK 2 – Add & View Requests

Prompt

Add GET and POST endpoints to manage service requests with title and priority using Flask.

Code:

Gemini

```python
from flask import Flask, jsonify, request

app = Flask(__name__)

# In-memory storage for service requests
requests = []
request_id_counter = 1

@app.route('/status')
def status():
    return jsonify({'status': 'ok', 'message': 'API is running'})

@app.route('/requests', methods=['POST'])
def create_request():
    global request_id_counter
    data = request.get_json()
    if not data or 'title' not in data or 'priority' not in data:
        return jsonify({'error': 'Missing title or priority'}), 400

    new_request = {
        'id': request_id_counter,
        'title': data['title'],
        'priority': data['priority']
    }
    requests.append(new_request)
    request_id_counter += 1
    return jsonify(new_request), 201

@app.route('/requests', methods=['GET'])
def get_requests():
    return jsonify(requests)

if __name__ == '__main__':
    # In Colab, you might need to use a different host to make it accessible
    # from the Colab environment. '0.0.0.0' is generally used for external access.
    # However, for local testing within Colab, '127.0.0.1' or even not specifying
    # the host might work depending on the Colab environment setup.
    # For a simple run, you can just use app.run(debug=True).
    # If you intend to expose it via ngrok or similar, you'd configure that separately.
    print("To run the app, execute 'python your_file_name.py' in a terminal if this were a script.")
    print("In Colab, you can run it directly, but accessing it from outside Colab cells might require port forwarding or ngrok.")
    app.run(host='0.0.0.0', port=5000, debug=True)
```

Output:

```
To run the app, execute 'python your_file_name.py' in a terminal if this were a script.
In Colab, you can run it directly, but accessing it from outside Colab cells might require port forwarding or ngrok.
 * Serving Flask app '__main__'
 * Debug mode: on
INFO:werkzeug:WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5000
 * Running on http://172.28.0.12:5000
INFO:werkzeug:Press CTRL+C to quit
INFO:werkzeug: * Restarting with watchdog (inotify)
```

Justification

Implements basic CRUD (Create + Read)

Uses in-memory storage (simple for lab)

TASK 3 – Update Resource (PUT)

Prompt

Add PUT endpoint to update a request by index with proper error handling in Flask.

Code:

```python
from flask import Flask, jsonify, request

app = Flask(__name__)

# In-memory storage for service requests
requests = []
request_id_counter = 1

@app.route('/status')
def status():
    return jsonify({'status': 'ok', 'message': 'API is running'})

@app.route('/requests', methods=['POST'])
def create_request():
    global request_id_counter
    data = request.get_json()
    if not data or 'title' not in data or 'priority' not in data:
        return jsonify({'error': 'Missing title or priority'}), 400

    new_request = {
        'id': request_id_counter,
        'title': data['title'],
        'priority': data['priority']
    }
    requests.append(new_request)
    request_id_counter += 1
    return jsonify(new_request), 201

@app.route('/requests', methods=['GET'])
def get_requests():
    return jsonify(requests)

@app.route('/requests/<int:request_id>', methods=['PUT'])
def update_request(request_id):
    data = request.get_json()
    if not data or 'title' not in data or 'priority' not in data:
        return jsonify({'error': 'Missing title or priority in request body'}), 400

    for req in requests:
        if req['id'] == request_id:
            req['title'] = data['title']
            req['priority'] = data['priority']
            return jsonify(req), 200
    return jsonify({'error': 'Request not found'}), 404

if __name__ == '__main__':
    # In Colab, you might need to use a different host to make it accessible
    # from the Colab environment. '0.0.0.0' is generally used for external access.
    # However, for local testing within Colab, '127.0.0.1' or even not specifying
    # the host might work depending on the Colab environment setup.
    # For a simple run, you can just use app.run(debug=True).
    # If you intend to expose it via ngrok or similar, you'd configure that separately.
    print("To run the app, execute 'python your_file_name.py' in a terminal if this were a script.")
    print("In Colab, you can run it directly, but accessing it from outside Colab cells might require port forwarding or ngrok.")
    app.run(host='0.0.0.0', port=5000, debug=True)
```

Output:

```
To run the app, execute 'python your_file_name.py' in a terminal if this were a script.
In Colab, you can run it directly, but accessing it from outside Colab cells might require port forwarding or ngrok.
 * Serving Flask app '__main__'
 * Debug mode: on
INFO:werkzeug:WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5000
 * Running on http://172.28.0.12:5000
INFO:werkzeug:Press CTRL+C to quit
INFO:werkzeug: * Restarting with watchdog (inotify)
```

+ Code + Text

Justification:

Enables update operation

Error handling prevents crashes

TASK 4 – Delete Resource

Prompt

Add DELETE endpoint to remove a request by index with safe error handling.

Code:

```python
from flask import Flask, jsonify, request

app = Flask(__name__)

# In-memory storage for service requests
requests = []
request_id_counter = 1

@app.route('/status')
def status():
    return jsonify({'status': 'ok', 'message': 'API is running'})

@app.route('/requests', methods=['POST'])
def create_request():
    global request_id_counter
    data = request.get_json()
    if not data or 'title' not in data or 'priority' not in data:
        return jsonify({'error': 'Missing title or priority'}), 400

    new_request = {
        'id': request_id_counter,
        'title': data['title'],
        'priority': data['priority']
    }
    requests.append(new_request)
    request_id_counter += 1
    return jsonify(new_request), 201

@app.route('/requests', methods=['GET'])
def get_requests():
    return jsonify(requests)

@app.route('/requests/<int:request_id>', methods=['PUT'])
def update_request(request_id):
    data = request.get_json()
    if not data or 'title' not in data or 'priority' not in data:
        return jsonify({'error': 'Missing title or priority in request body'}), 400

    for req in requests:
        if req['id'] == request_id:
            req['title'] = data['title']
            req['priority'] = data['priority']
            return jsonify(req), 200
    return jsonify({'error': 'Request not found'}), 404

@app.route('/requests/<int:request_id>', methods=['DELETE'])
def delete_request(request_id):
    global requests
    original_length = len(requests)
    requests = [req for req in requests if req['id'] != request_id]
    if len(requests) < original_length:
        return jsonify({'message': f'Request with id {request_id} deleted successfully'}), 200
    return jsonify({'error': 'Request not found'}), 404

if __name__ == '__main__':
    # In Colab, you might need to use a different host to make it accessible
    # from the Colab environment. '0.0.0.0' is generally used for external access.
    # However, for local testing within Colab, '127.0.0.1' or even not specifying
    # the host might work depending on the Colab environment setup.
    # For a simple run, you can just use app.run(debug=True).
    # If you intend to expose it via ngrok or similar, you'd configure that separately.
    print("To run the app, execute 'python your_file_name.py' in a terminal if this were a script.")
    print("In Colab, you can run it directly, but accessing it from outside Colab cells might require port forwarding or ngrok.")
    app.run(host='0.0.0.0', port=5000, debug=True)
```

Output:

```
To run the app, execute 'python your_file_name.py' in a terminal if this were a script.
In Colab, you can run it directly, but accessing it from outside Colab cells might require port forwarding or ngrok.
 * Serving Flask app '__main__'
 * Debug mode: on
INFO:werkzeug:WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5000
 * Running on http://172.28.0.12:5000
INFO:werkzeug:Press CTRL+C to quit
INFO:werkzeug: * Restarting with watchdog (inotify)
```

Justification

- Completes CRUD (Delete)
- Safe removal with validation

TASK 5 – Documentation (AI Assisted)

Prompt

Add docstrings and comments to Flask API endpoints and optionally integrate Swagger documentation.

Code:

```python
from flask import Flask, jsonify, request

app = Flask(__name__)

# In-memory storage for service requests
requests = []
request_id_counter = 1

@app.route('/status')
def status():
    """
    GET /status
    Returns the API status.

    Responses:
        200: {'status': 'ok', 'message': 'API is running'}
    """
    return jsonify({'status': 'ok', 'message': 'API is running'})

@app.route('/requests', methods=['POST'])
def create_request():
    """
    POST /requests
    Creates a new service request.

    Request Body (JSON):
        {'title': 'string', 'priority': 'string'}

    Responses:
        201: New request object with 'id', 'title', 'priority'
        400: {'error': 'Missing title or priority'}
    """
    global request_id_counter
    data = request.get_json()
    # Validate input data
    if not data or 'title' not in data or 'priority' not in data:
        return jsonify({'error': 'Missing title or priority'}), 400

    # Create a new request object
    new_request = {
        'id': request_id_counter,
        'title': data['title'],
        'priority': data['priority']
    }
    requests.append(new_request)
    request_id_counter += 1
    return jsonify(new_request), 201

@app.route('/requests', methods=['GET'])
def get_requests():
    """
    GET /requests
    Retrieves all service requests.

    Responses:
        200: A list of all request objects.
    """
    return jsonify(requests)

@app.route('/requests/<int:request_id>', methods=['PUT'])
def update_request(request_id):
    """
    PUT /requests/<request_id>
    Updates an existing service request by its ID.

    Path Parameters:
        request_id (int): The ID of the request to update.
```

es Terminal

```python
    # Validate input data
    if not data or 'title' not in data or 'priority' not in data:
        return jsonify({'error': 'Missing title or priority in request body'}), 400

    # Find and update the request
    for req in requests:
        if req['id'] == request_id:
            req['title'] = data['title']
            req['priority'] = data['priority']
            return jsonify(req), 200
    return jsonify({'error': 'Request not found'}), 404

@app.route('/requests/<int:request_id>', methods=['DELETE'])
def delete_request(request_id):
    """
    DELETE /requests/<request_id>
    Deletes a service request by its ID.

    Path Parameters:
        request_id (int): The ID of the request to delete.

    Responses:
        200: {'message': 'Request with id {request_id} deleted successfully'}
        404: {'error': 'Request not found'}
    """
    global requests
    original_length = len(requests)
    # Filter out the request to be deleted
    requests = [req for req in requests if req['id'] != request_id]
    if len(requests) < original_length:
        return jsonify({'message': f'Request with id {request_id} deleted successfully'}), 200
    return jsonify({'error': 'Request not found'}), 404

if __name__ == '__main__':
    # In Colab, you might need to use a different host to make it accessible
    # from the Colab environment. '0.0.0.0' is generally used for external access.
    # However, for local testing within Colab, '127.0.0.1' or even not specifying
    # the host might work depending on the Colab environment setup.
    # For a simple run, you can just use app.run(debug=True).
    # If you intend to expose it via ngrok or similar, you'd configure that separately.
    print("To run the app, execute 'python your_file_name.py' in a terminal if this were a script.")
    print("In Colab, you can run it directly, but accessing it from outside Colab cells might require port forwarding or ngrok.")
    app.run(host='0.0.0.0', port=5000, debug=True)
```

Output:

```
To run the app, execute 'python your_file_name.py' in a terminal if this were a script.
In Colab, you can run it directly, but accessing it from outside Colab cells might require port forwarding or ngrok.
 * Serving Flask app '__main__'
 * Debug mode: on
INFO:werkzeug:WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5000
 * Running on http://172.28.0.12:5000
INFO:werkzeug:Press CTRL+C to quit
INFO:werkzeug: * Restarting with watchdog (inotify)
```

Justification

- Docstrings → improve readability
- Swagger → auto API documentation
- Helps other developers integrate faster